

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been copied and communicated to you by or on behalf of The University of Newcastle pursuant to Part VB of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

SENG1050

Web Technologies

Week 6 - The Client-Server Model

Assessment 1 (Mid-Semester Quiz)

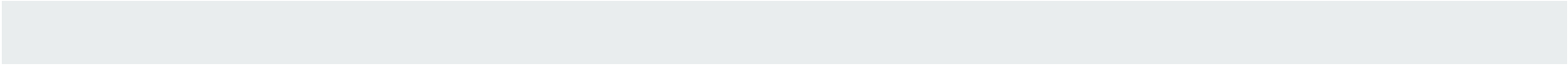
- ❑ Takes place on 12 Sep (Friday next week). The online quiz will open from 8 am to 11:59 pm.
- ❑ Assessment time: 90 minutes (answering) + 10 minutes (checking). All quiz sessions will end at 11:59 pm, so if you start the quiz after 10:19 pm, you won't get the full time.
- ❑ Access via Canvas – Modules -> Assessments -> Mid-Semester Quiz. The quiz is open-book, however, all answers must be in your own words. Use of generative AI (such as ChatGPT) and plagiarism is strictly forbidden (all submissions will be checked).

Quiz format & coverage

- ❑ **Coverage:** Weeks 1-5.
- ❑ 15 questions (30 including sub-questions): mix of matching, multiple choice and short answer/essay (may include simple coding).
- ❑ Study slides marked with stars.

Lecture Outline

- ❑ The Client-Server Model
- ❑ Apache
- ❑ PHP



Recall from last week: Advanced HTML and CSS

- ❑ We completed the separation of styling and structure using a CSS reset to clear browser default styles. CSS positioning and display properties now allow us to create our own custom page layouts.
- ❑ We also learned some more advanced dynamic features of HTML such as image maps, tables, and forms to display complex data, and receive data from the client.
- ❑ Two problems have become apparent: first, what happens when the client submits a form? At the moment either nothing happens, or we redirect the client to a test script that does nothing but echo the submitted data back to the client.
- ❑ The second problem has been growing more apparent over the past few weeks: all of our websites so far have only been viewable on our own computer, how do we make our website accessible on the internet?
- ❑ To address both problems, today we will turn our attention to web servers, what they do, how they are setup, and how we can customise their behaviour.

The Client-Server Model

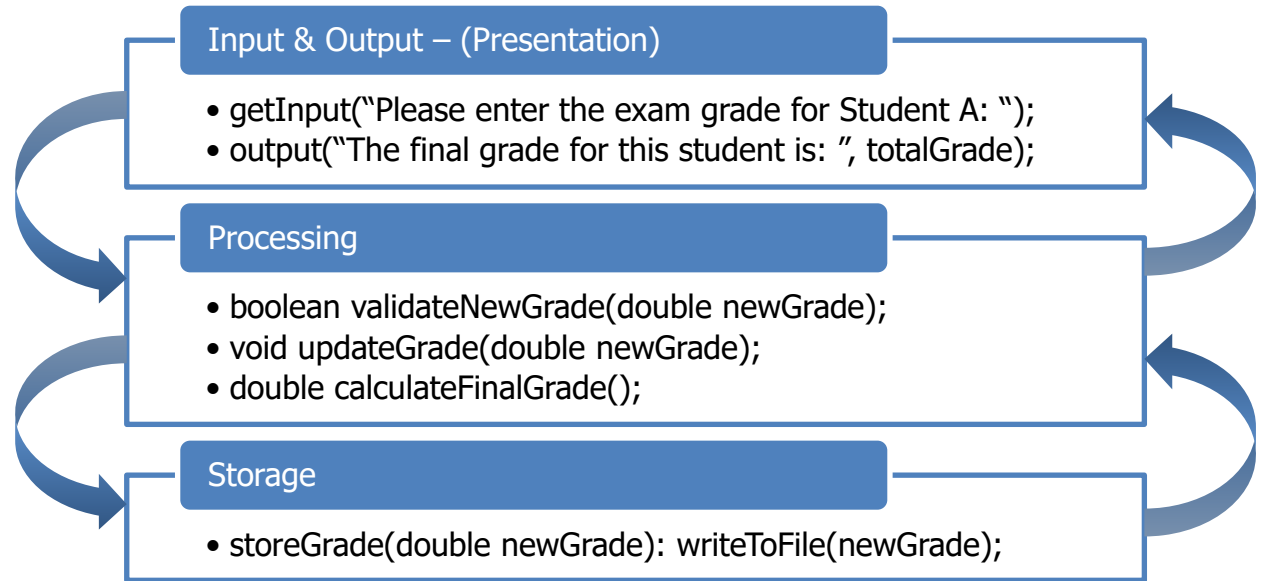


What is a Service?

- ❑ In networking terms, a 'service' is a defined set of functionality that is made accessible within a specific network.
- ❑ Examples of common services are printing, file storage, email, multiplayer gaming, and remote device access.
- ❑ The fundamental challenge of network services is that, unlike traditional desktop applications, the functionality is split across multiple devices that need to communicate with each other to ensure that the service operates correctly.
- ❑ The functionality of a given application can be grouped into four broad categories: input, processing, storage, and output. For large applications, these categories can be subdivided even further.

Web Services vs. Desktop Applications

Desktop Application



Web Services vs. Desktop Applications

Web Service



Input & Output – (Presentation)

- getInput("Please enter the exam grade for Student A: ");
- output("The final grade for this student is: ", totalGrade);

Processing

- boolean validateNewGrade(double newGrade);
- void updateGrade(double newGrade);
- double calculateFinalGrade();

Storage

- storeGrade(double newGrade):
- writeToFile(newGrade);



Clients and Servers

- ❑ The consumer (user) of a service is called the 'client':
 - The term client can refer to **hardware** (e.g., multiple physical computers accessing a print service).
 - The term client can also refer to **software** (e.g., Your web browser accessing the audio service provided by the operating system to play a video)
- ❑ The provider of a service is called the 'server'.
 - A server **in hardware terms** is a host on a network that provides a service to other hosts on the network. This usually refer to specialised computers with high-performance hardware designed to handle the unique needs of providing services.
 - **In software terms**, a server is an application that provides a service to other applications on the same network. This network can be the Internet (a network of networks).
- ❑ Clients request the service from the server, which will respond to (serve) the requests. Typically, a single server will provide a service to multiple clients.



Distributed Applications

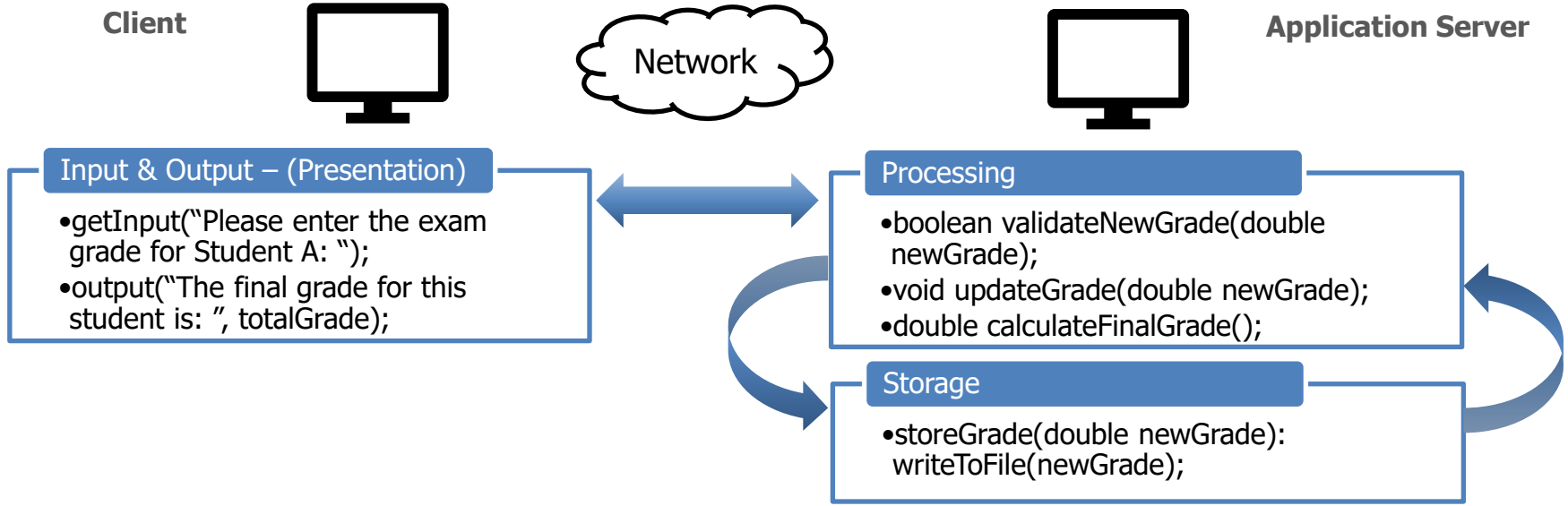
- ❑ Distributing parts of an application among different devices introduces some unique challenges that each application needs to solve.
- ❑ There are many challenges that a distributed application may need to address, however the four most common are:
 - Scalability – A service may initially have only a few clients, but in future become popular and attract an increasing number of clients. The amount of data that needs to be managed is also likely to grow over time. These factors contribute to an increased demand for resources by each device involved in providing the service.
 - Fault Tolerance – What should the application do if communication with any single given device fails? Failures can occur in many places such as devices, communication links, and software. Detecting and responding to each different type of failure is essential to ensure the stability of the service.
 - Concurrency – This addresses the problem of different devices holding different data, such as slightly different clock times. It also considers the question of what should the application do if multiple devices attempt to modify the same data at the same time.
 - Security – There are several factors that need to be considered, such as preventing message interception, preventing device impersonation, and preventing message modification in transit.



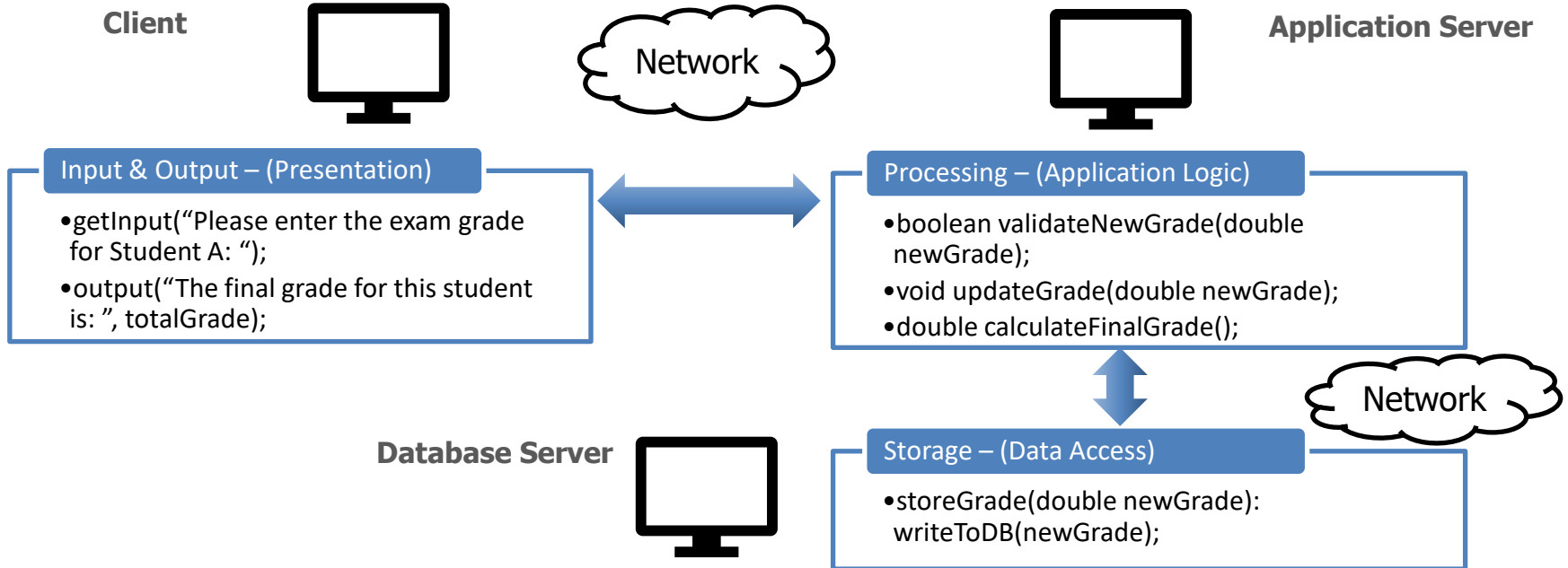
Distributed Architectures

- ❑ There are many ways to solve the challenges presented by distributed applications, each with their own advantages and disadvantages. The two most common solutions are the client-server model, and the peer-to-peer model.
- ❑ The **client-server model** creates a **clear distribution of responsibilities** between devices. There is a dedicated device (or devices) responsible for providing the service to clients. These devices, or servers, are responsible for storing and managing access to data according to a set of pre-defined rules. The clients are responsible for initiating communication with the servers, who will respond to these requests. Clients are not expected to contribute their own resources to ensuring the stability of the service. Compared to other hosts on a network, servers are generally expected to be more reliable, to have minimal downtime (in some cases even no downtime), and to implement a greater degree of security.
- ❑ The **peer-to-peer model** removes the delineation between clients and servers, allowing **each device to act as both**. In this way, each device contributes part of its own resources to providing the service, and allows for decentralised administration.

The Client-Server Model



The Client-Server Model





Why the Client-Server Model?

- ❑ The client-server model is designed to **centralize data storage** on a network, this makes backup and recovery easier as a smaller number of devices need to be managed. Further, **data integrity** can be more easily enforced, as there will be no conflicting versions of the data, with the server acting as the single source of truth.
- ❑ Servers can be designated as trusted devices and assigned the role of handling **data security, access control, and logging** access to data. This reduces the risk of a malicious actor gaining access to data via compromising a trusted client. All clients are generally designated as untrusted, the server will handle all operations involving sensitive data. Devices designated as servers in this model must therefore implement a greater degree of security.
- ❑ Devices that are dedicated to acting as servers are usually equipped with higher-performance hardware than clients, and this can be leveraged by **performing processor-intensive tasks** such as pre-processing and dynamic content generation on the server, eliminating the need for the client to make multiple comparatively slower requests to the server over the network to perform the same task.



Why Not the Client-Server Model?

- ❑ For every advantage provided by the client-server model, there is a corresponding disadvantage.
- ❑ While centralisation of data is good for ensuring data integrity, it does have the downside of creating a **single point of failure**. If the server fails, the entire service becomes inoperable.
- ❑ Similarly, while the zero-trust policy implemented by the client-server model is good for preventing attacks by malicious clients, it also creates a single point of weakness to an active attacker. If a malicious actor can successfully infiltrate the server, all clients using the service become vulnerable.
- ❑ While performing processing and content generation on the server does remove workload from the client and reduce network traffic, it also creates a **performance bottleneck** as the server must have the resources to keep up with all the incoming requests. This is also a security vulnerability – e.g., denial of service attacks.



Why the Peer-to-Peer Model?

- ❑ The peer-to peer model is an alternative to the client-server model that decentralises the service among each connected host or 'peer'. Peers contribute a portion of their resources to providing the service, which provides a **huge scalability advantage** when service usage increases, as the resources available to the service automatically grow with the number of peers using the service.
- ❑ The peer-to-peer model is far more **fault tolerant** compared to the client-server model, as the service can continue to operate with the loss of a single peer (or even several peers). This has caused some consternation for authorities when the service in question is illegal (such as filesharing of movies – e.g., Napster, Grokster, etc) as peers are often in different countries, and arresting one person (removing a single peer) is not enough to prevent access to the service.
- ❑ Unlike the client-server model, **infiltrating a single device is not enough to compromise all peers** connected to a service using the peer-to-peer model. If there is a conflict about which source of data is correct, peers can communicate among each other to form a democratic consensus, so a single bad actor is quickly identified.



Why Not the Peer-to-Peer Model?

- ❑ The decentralised nature of the peer-to-peer model can be **unfeasible** for many applications, **when centralised control of data is required**. For example, secret information cannot be shared among multiple connected peers, but must be kept secure with access controls in place.
- ❑ The peer-to-peer model uses **far more bandwidth** compared to the client-server model, as peers are constantly communicating with each other. As a result of this, some ISPs have attempted to intentionally slow traffic identified as being associated with a peer-to-peer service as it consumes large portions of their bandwidth, increasing costs.
- ❑ Peer-to-peer applications are generally **more complex**, as peers are highly likely to have differing sets of data, requiring additional concurrency controls **to ensure data integrity**.

Server Types

- ❑ To maximise performance, servers do not run any applications beyond what is required for them to perform their function. Unnecessary applications are removed, increasing performance and minimising the potential attack surface for a malicious actor.
- ❑ Common types of servers include:
 - ❑ Catalog server – maintains a catalog or directory of information that can be found and searched on the network. For example, Domain Directory Services (such as Active Directory) are provided by a catalog server.
 - ❑ Database server – maintains a database for access over the network, such as for accounting programs, client relationship management (CRM) programs, business intelligence analytics, etc.
- ❑ File server – provides a large, centralised storage space for files and folders over a network.
- ❑ Mail server – provides email services to a network (usually as part of a domain).
- ❑ Proxy server – Sends and receives network traffic on behalf of another host or hosts on a network. Can be used to implement several functions such as content filtering, antivirus scanning, and caching.
- ❑ Web server – Hosts content for one or more websites as part of the World Wide Web.
- ❑ A single server may provide several functions, implementing more than one of the above roles.

Client Types

- ❑ As servers become faster, and modern computing increasingly moves online using technologies such as cloud computing, the need for high-performance clients, particularly in the enterprise, has decreased over time. Different types of clients can be used to perform computing tasks under the client-server model:
 - ❑ **Thin clients** are simple computing devices with lower-performance, cheaper hardware. They are mostly used to connect to servers where the majority of tasks are performed. Generally only the interface component of an application is implemented on a thin client.
 - ❑ **Thick clients** use higher-performance hardware, designed to handle most processing themselves without relying heavily on the server. They can handle the control logic of most applications, only using the server to send and receive shared data from a database.
 - ❑ **Hybrid clients** are a mix of thick and thin clients, using the server for data storage, but handling certain processing themselves.

Web Servers

- ❑ A web server is an application that hosts content for one or more websites, as part of the World Wide Web.
- ❑ The web server listens to network traffic, usually on ports 80 and 443 (HTTP and HTTPS), receiving and processing incoming requests, and responding with the requested content (HTML, CSS, images, etc).
- ❑ During operation the web server manages active connections, closing connections and accepting new connections depending on the resources available.

Using a Web Server

- ❑ Modern web servers include Microsoft's Internet Information Services (IIS), Apache, and Nginx, which together currently handle over 60% of website traffic.
- ❑ A web server is generally not sufficient on its own for most modern applications, it needs some data storage solution (such as a database), and some server-side behaviour component.
- ❑ This is commonly implemented as part of a web architecture stack, such as LAMP (Linux, Apache, MySQL, PHP) or MEAN (MongoDB, ExpressJS, AngularJS, NodeJS). For this course we will use a modified version of the LAMP stack, replacing Linux with the more familiar Windows (WAMP). Additionally, using XML files instead of a database means we really only need to consider two components in depth – Apache and PHP.
- ❑ For added simplicity, we will use the software package XAMPP, which will allow us to deploy and manage the entire stack all at once.

Which of the following is NOT an advantage to using the client-server model?

Security is increased by barring the client from performing any trusted operations.

0%

The decentralization of data means increased reliability as there is no single point of failure for the servi...

0%

Page load is improved through pre-processing on the faster server hardware.

0%

Disaster recovery is faster as fewer devices need to be restored after data loss.

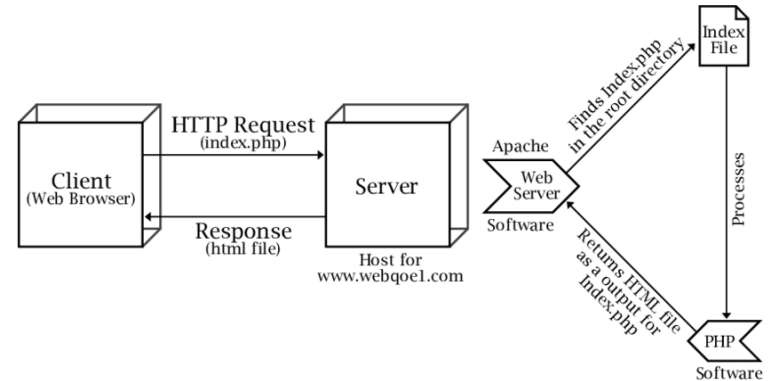
0%



Apache

The Apache Web server

- ❑ Apache is a commonly used open-source web server that currently serves around 30% of all website traffic on the internet. Developed in 1995, it is highly customisable, and is one of the most secure web servers in use.
- ❑ Apache is very extensible, with multiple modules available that add additional functionality to the core web server, such as SSL support, caching, custom authentication, compression, and more.





Setup

- ❑ When Apache is first installed, it will create a directory on the host for serving webpages. This is the called the 'document root'. By default, this will be inside the installation folder for Apache, in a directory called 'htdocs'. Using XAMPP, it will be again be inside the installation folder, in a directory called 'xamp/htdocs'.
- ❑ Any files placed inside the document root will be accessible to any visitors to your website. By default, Apache will look for a file called 'index.html' in any directory requested by a client, but this can be changed in the configuration (for example, to 'index.php').
- ❑ Apache has the capability to host pages for multiple websites. Each website is contained in a virtual host with its own document root and configuration.

Virtual Hosts

- ❑ A virtual host differs from a physical host, in that it is contained on a physical host. Multiple virtual hosts can exist on an individual physical host, each one serving content for a different website.
- ❑ The default Apache installation comes with a virtual host already setup.
- ❑ Using XAMPP, you can create more virtual hosts. We will practice this in Lab 6.

Common Security Configurations

- ❑ When Apache is started, settings are loaded from a configuration file, called 'httpd.conf'. This file, as well as files that are included as part of it, can be edited to change the default behaviour of the web server.
- ❑ For example, Apache includes its name and version, as well as the name and version of the host operating system in every response it gives to a client, potentially compromising security by allowing an attacker to target zero-day vulnerabilities in a specific version of the software. To solve this, we can turn off the server signature by changing the following lines in the configuration file:
- ❑ Secondly, a client requesting a directory without a default HTML file ('index.html') will usually get a directory listing of all files contained within the directory. As this can unintentionally reveal information about the structure of your website, we can turn this off in the configuration by looking for the 'Options' inside the 'Directory' matching the document root, and changing '+Indexes' to '-Indexes'.

```
'ServerSignature On' change to 'ServerSignature Off'  
'ServerTokens Full' change to 'ServerTokens Prod'
```

Putting Everything Together - Live Demo

- ❑ Refer to the [supplementary video in Lab 6](#) which shows how a web server is deployed on a virtual machine hosted on Microsoft Azure. You will go through the same steps in Lab 6 Task 1.
- ❑ Outside the classroom, a basic web server can be setup using the free tier on Google Cloud to create a virtual machine running Linux. We will install Apache on the server, and then setup DNS to point to the server.
- ❑ Other methods (including paid hosting and other services) can be setup in a similar way.

Your Turn (In Lab 6)

- ❑ You should be able to install XAMPP on your personal computer or laptop to work on upcoming labs and Assessment 3. In general, Assessment 3 will require you to migrate your website to the Apache webserver.
- ❑ You have also been sent an email (with [registration link](#)) to a Microsoft Azure virtual machine with XAMPP and Notepad++ pre-installed. This virtual machine will be a backup in case you could not install XAMPP on your personal computer.
- ❑ After installing XAMPP/registering for the virtual machine, you will perform the setup presented in Lab 6. Remember to follow the supplementary video if you are stuck.

PHP

PHP Hypertext Preprocessor

- ❑ Recall from Week 2: HyperText is “A body of written or pictorial material [emphasis on text] interconnected in a complex way that it could not be conveniently represented on paper. It may contain summaries or maps of its contents and their interrelations it may contain annotations, additions, and footnotes from scholars who examined it.” – Ted Nelson
- ❑ A pre-processor is a program designed to transform source data before it is passed to another program (such as a compiler).
- ❑ PHP is a server-side scripting language used for implementing custom behaviour in response to a HTTP request, taking the source data from multiple locations such as XML files, database queries, HTML templates, etc.
- ❑ Over 75% of currently active websites use PHP either entirely or as part of their server-side programming.

PHP Hypertext Preprocessor

- ❑ PHP is used to implement a wide variety of custom functionality for websites, such as form handling, access control, custom searching, creating dynamic content, and much more.
- ❑ A PHP script running on the server can read and write files on the server, query and write to any running databases (most commonly some form of SQL database such as MySQL), and access the content and header information of the initiating HTTP request during execution.
- ❑ By default, the output of a PHP script is used by the web-server as the body of the HTTP response, the most common output is a HTML document, or a redirection (using a 3xx response code).



Basic Syntax

- ❑ A PHP script begins with `<?php` and ends with `?>`, a single file can contain multiple PHP scripts intermixed with HTML markup. The file itself does not have to start with `<?php` this only needs to come before the first PHP statement.
- ❑ Each statement in PHP must end with a semicolon ;
- ❑ To aid readability, PHP scripts should be commented:
 - ❑ A single-line comment begins with either `//` or `#`.
 - ❑ A multi-line comment begins with `/*` and ends with `*/`



Output

- ❑ The output of PHP scripts is used by the web-server as the body of the HTTP response, therefore any output should usually consist of valid HTML.
- ❑ To output from a script, use the `echo` statement or the `print` statement.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>My First PHP Script</title>
  </head>
  <body>
    <?php
      echo "<p>Hello World</p>";
    ?>
  </body>
</html>
```



Hello World



Variables

```
<?php
    $name = "Alice";
    $age = 22;
    $enrolled = true;
?>
```

- ❑ A variable is used to store data needed by a script. A variable is a container for information that you want to store. A variable's value can change during the script.
- ❑ Variables can store any kind of data: numbers, decimals, text, true/false values (also known as Boolean values), and many others.
- ❑ You can name your variables anything you like, but it is good practice to give names that indicate the purpose of each variable. You can use this name to see or change the value.
- ❑ A variable in PHP starts with a **\$** followed by the name of the variable. Variable names can only contain letters, digits, and the underscore `_` character. They cannot begin with a digit.
- ❑ Variables in PHP are case sensitive, so `$name` and `$NAME` will be treated as separate variables.



Form Input

```
<form method="POST" action="my_script.php">
    <input type="text" name="fname" /><br>
    <input type="text" name="lname" />
</form>
```

- ❑ A PHP script has access to the content of the initiating HTTP request, this can include data submitted via a form.
- ❑ Remember from last week: each input element in a form should have a **name** attribute set. We can now use the values of these attributes to retrieve the values for our script:
- ❑ The **\$_GET** and **\$_POST** arrays contain the submitted data, depending on the HTTP request method used.

```
<?php
    $first_name = $_POST["fname"];
    $last_name = $_POST["lname"];
    echo "<p>Your name is" . $first_name . " " . $last_name . "</p>";
?>
```



Strings

- ❑ Text content in a variable is referred to as a 'string'.
- ❑ Strings should be inside single or double quotes e.g. "This is a string". This is important if a number needs to be treated as text, i.e. variables `$age = 23`; `$age = "23"`; will behave differently.
- ❑ Strings can be joined together (concatenated) with the use of the dot `.` operator:

```
<?php
    $first_name = $_POST["fname"];
    $last_name = $_POST["lname"];
    echo "<p>Your name is" . $first_name . " " . $last_name . "</p>";
?>
```

String Methods

❑ PHP contains several in-built methods to perform common operations with strings:

- ❑ `strlen()` – Get the length of a string
- ❑ `strtolower()` – Convert a string to lowercase
- ❑ `strtoupper()` – Convert a string to uppercase
- ❑ `strpos()` – Search for a string inside another string
- ❑ `str_replace()` – Replace part of a string with another string
- ❑ `trim()` – Remove whitespace from the beginning and end of a string

```
<?php
    $my_string = "  Hello World!  ";
    $trimmed_string = trim($my_string);
    echo strtolower($my_string); //hello world!
    echo strtoupper($my_string); //HELLO WORLD!
    echo strlen($trimmed_string); //12
    echo strpos($trimmed_string, "World"); //6 (start counting at zero, not one)
    echo str_replace("l", "*", $trimmed_string); //He**o Wor*d!
?>
```



Arithmetic

```
<?php
    $a = 5;
    $b = 10;
    $c = $a + $b; //15
    echo $c - $b; //5
    echo $a * $b; //50
    echo $b / $a; //2
    echo $b % $c; //10 (The remainder of 10 divided by 15)
    echo $a ** 2; //25 (5 raised to the power of 2)
?>
```

- ❑ Many scripts involve performing some form of calculation using numbers.
- ❑ We will refer to whole numbers as 'integers', and decimal numbers as 'floats' (also called 'floating point numbers').
- ❑ PHP supports 6 standard arithmetic operations: addition, subtraction, multiplication, division, division remainder (modulo), and power (exponentiation). The operators are **+**, **-**, *****, **/**, **%**, and ****** respectively.

Arithmetic Methods

- ❑ PHP includes several built-in methods for common mathematical operations:

- ❑ `sqrt()` – Calculate the square root of a number.
- ❑ `abs()` – Calculate the absolute value of a number.
- ❑ `min()` – Find the smallest value from of a set of numbers.
- ❑ `max()` – Find the largest value from a set of numbers.
- ❑ `round()` – Round a number to the nearest integer.
- ❑ `rand()` – Generate a random number.

```
<?php
    $a = 25;
    $b = -5;
    $c = 9.1;
    echo sqrt($a); //5
    echo abs($b); //5
    echo min($a, $b, $c); //-5
    echo max($a, $b, $c); //25
    echo round($c); //9
    echo rand(0, 100); // Some random number between 0 and 100 inclusive
?>
```



Conditions

- ❑ Most scripts involve branch selection – executing different code based on whether or not a particular condition is met.
- ❑ Conditions can either be met or not met, that is, they are true or false (a Boolean value).
- ❑ Conditions are written using comparison operators.

==	Equal to	5 == 5 (True)
!=	Not equal to	4 != 5 (True)
<	Less than	3 < 1 (False)
>	Greater than	6 > 6 (False)
<=	Less than or equal to	4 <= 4 (True)
>=	Greater than or equal to	6 >= 7 (False)



'if' Statements

- ❑ We can use a condition with an 'if' statement to make part of a PHP script optional.
- ❑ The body of an if statement is only executed if the condition evaluates to TRUE. There is no limit to the length of the body of an if statement.

- ❑ The syntax of an if statement is:

```
if (<condition>) {  
    // Code here will only execute if the condition is true  
}
```

- ❑ For example:

```
if ($age < 18) {  
    echo "Error, you must be 18 or older to rent this movie";  
}
```



'if-else' Statements

- ❑ Sometimes we want to execute one branch of code if a condition is true, and a different branch if that same condition is false.
- ❑ We can use an 'if-else' statement to achieve this. The syntax for an if-else statement is:

```
if (<condition>) {  
    // This executes if the condition is true  
}  
else {  
    // This executes if the condition is false  
}
```

```
if ($price > 100) {  
    $delivery_fee = 0;  
}  
else {  
    $delivery_fee = 30;  
    echo "<p>A delivery fee of $30 applies to your purchase</p>";  
}
```

'if-elseif-else' Statements

- ❑ This can be extended to an infinite number of conditions using the if-elseif-else statement. Only one block in the statement will run, depending on which condition (if any) is true.
- ❑ An unlimited amount of elseif blocks can be added to the basic if statement. An else block is required, this will run only if none of the conditions are true.

```
if (<condition_1>) {  
    // This runs if condition 1 is true  
}  
elseif (<condition_2>) {  
    // This runs if condition 1 is false and  
    //condition 2 is true  
}  
elseif (<condition_3>) {  
    // This runs if conditions 1 and 2 are false,  
    //and condition 3 is true  
} // More elseif blocks can be added  
else {  
    // This runs if none of the previous  
    //conditions are true  
}
```

Conditional Assignment

- ❑ We can use conditions to assign values to variables using the ternary operator.
- ❑ The ternary operator assigns one of two values to a variable depending on whether or not a condition is met.
- ❑ The syntax for the ternary operator is:
`$variable = <condition> ? <value_if_true> : <value_if_false>;`
- ❑ For example:
`$discount = $membership_years > 5 ? 20 : $membership_years * 2;`



Looping

++	Increment	\$x++
--	Decrement	\$x--

- ❑ Aside from branch selection, the second type of common programming structure is called 'iteration'. That is, repeating or 'looping' a series of statements multiple times. This can save us from having to write the same statement multiple times.
- ❑ When using loops, **we need to be careful to avoid our script going on forever, never ending**. We need to know when to stop a loop. We will do this by checking a condition each time the loop runs. When the condition is false, we will stop the loop and continue with the next statement in the script.
- ❑ We usually need to change the variable in the condition inside the loop. This is commonly done using the increment and decrement operators, though any operations can be used.



'while' loop

- ❑ The simplest type of loop is the 'while' loop. This uses a single condition followed by the body of the loop.
- ❑ At the start of each loop, the condition is checked. When the condition is false, the loop will stop.

The syntax for the 'while' loop is:

```
while (<condition>) {  
    // Body of the loop, this runs as long as the condition is true.  
}
```

- ❑ For example:

```
$countdown = 10;  
while ($countdown > 0) {  
    echo "<p>The countdown is: " . $countdown . "</p>";  
    $countdown--;  
}
```

The countdown is: 10

The countdown is: 9

The countdown is: 8

The countdown is: 7

The countdown is: 6

The countdown is: 5

The countdown is: 4

The countdown is: 3

The countdown is: 2

The countdown is: 1

'do-while' loop

- ❑ If we know the loop must run at least once, we can use a 'do-while' loop. This is exactly the same as the 'while' loop, except that the condition is checked at the end of the loop instead of the beginning. This means that the loop will always execute at least once:

```
do {  
    // Loop body  
} while (<condition>);
```



'for' loop

- ❑ Often, when dealing with loops we find ourselves using a variable setup specifically for the loop, and changing the variable either at the beginning or end of the loop.
- ❑ The 'for' loop is a powerful looping structure that combines all three elements into the head of the loop. The syntax is:

```
❑ for (<initialize variable>; <condition>;  
    <update>) {  
    // Loop body  
}
```
- ❑ The 'initialize variable' is an assignment statement that runs once before the first iteration of the loop.
- ❑ The condition is checked at the start of each iteration of the loop. If it is false, the loop ends.
- ❑ The 'update' expression runs at the end of each iteration of the loop. This usually updates the variable used in the condition.

'for' loop

```
<?php  
  
    $category_count = 5;  
    echo "<ol>";  
    for ($counter = 1; $counter <= $category_count; $counter++) {  
        echo "<li><a href='category' " . $counter . ".html>Category " . $counter . "</a></li>";  
    }  
    echo "</ol>";  
  
?>
```



```
<ol>  
<li><a href="category1.html">Category 1</a></li>  
<li><a href="category2.html">Category 2</a></li>  
<li><a href="category3.html">Category 3</a></li>  
<li><a href="category4.html">Category 4</a></li>  
<li><a href="category5.html">Category 5</a></li>  
</ol>
```

Stopping Loops Early

- ❑ Sometimes, we don't know in advance how many times a loop should run, or we may need to stop a particular iteration of a loop early.
- ❑ PHP gives us two statements to achieve this:
 - ❑ The **continue** statement: this immediately terminates the current iteration of the loop, beginning the next iteration (after checking the condition).
 - ❑ The **break** statement: this immediately terminates the entire loop, skipping to the next statement in the script.

Stopping Loops Early

```
<?php
    for ($counter = 10; true; $counter--) // This loop will run forever unless we use a break statement
    {
        if ($counter % 2 == 0) {continue;} // If counter is an even number, stop the current iteration
        elseif ($counter <= 0) {break;}    // Stop the loop when the counter reaches 0 or lower
        else {echo "<p>The counter is: " . $counter;}
    }
?>
```



The counter is: 9
The counter is: 7
The counter is: 5
The counter is: 3
The counter is: 1



Debugging and Validation

- ❑ If your script has errors, it will not execute. An error message will be sent to the client, which can be viewed in the inspector (in the console tab).
- ❑ When debugging, syntax errors can be found using an online validator, such as <https://phpcodechecker.com/>
- ❑ Logic errors (code that is technically correct, but produces incorrect output) are a bit harder. A common way of tracing these is by adding temporary echo statements that output important variables at different points in the code.

Debugging and Validation

❑ This script runs forever even though we used a break statement, what's going on?

```
<?php
    for ($counter = 10; true; $counter--) // This loop will run forever unless we use a break statement
    {
        if ($counter % 2 == 0) {continue;}
        elseif ($counter == 0) {break;}
        else {echo "<p>The counter is: " . $counter;}
    }
?>
```

❑ Let's debug this by adding some extra outputs:

Debugging and Validation

- ❑ Looking at the output with debug statements, the problem is now clear.
- ❑ The loop should break when \$counter is zero, but the first condition also matches, and the loop iteration ends before the second condition is checked.
- ❑ There are a couple of ways to fix this, the simplest is just to switch the order of the conditions.

```
<?php
for ($counter = 10; true; $counter--)
{
    echo "Start Loop Iteration";
    if ($counter % 2 == 0) {
        echo "Continue: " . $counter; //DEBUG
        continue;
    }
    elseif ($counter == 0) {
        echo "Break: " . $counter; //DEBUG
        break;
    }
    else {
        echo "<p>The counter is: " . $counter;
    }
}
?>
```



Start Loop Iteration
Continue: 10
Start Loop Iteration
The counter is: 9
Start Loop Iteration
Continue: 8
The counter is: 7
Start Loop Iteration
Continue: 6
Start Loop Iteration
The counter is: 5
Start Loop Iteration
Continue: 4
The counter is: 3
Start Loop Iteration
Continue: 2
The counter is: 1
Start Loop Iteration
Continue: 0
Start Loop Iteration
The counter is: -1
Start Loop Iteration
Continue: -2 //etc

Debugging and Validation

- ❑ Looking at the output with debug statements, we can now see that the loop breaks at the correct point, and the problem is fixed.
- ❑ Don't forget to remove the debug statements before you submit your assignment!

```
<?php
    for ($counter = 10; true; $counter--)
    {
        echo "Start Loop Iteration";
        if ($counter == 0) {
            echo "Break: " . $counter; //DEBUG
            break;
        }
        elseif ($counter % 2 == 0) {
            echo "Continue: " . $counter; //DEBUG
            continue;
        }
        else {
            echo "<p>The counter is: " . $counter;
        }
    }
?>
```



Start Loop Iteration
Continue: 10
Start Loop Iteration
The counter is: 9
Start Loop Iteration
Continue: 8
The counter is: 7
Start Loop Iteration
Continue: 6
Start Loop Iteration
The counter is: 5
Start Loop Iteration
Continue: 4
The counter is: 3
Start Loop Iteration
Continue: 2
The counter is: 1
Start Loop Iteration
Break: 0



Debugging and Validation

- ❑ You should also check that the HTML documents generated by your scripts pass the HTML validator.
- ❑ To do this, open the inspector on the webpage returned by your script, right click on the root HTML element, select 'copy', and paste the contents into the HTML validator: <https://validator.w3.org/>

Which of the following PHP statements will output the following line: "Thank you for submitting the form"?

`echo "Thank you for submitting the form"`

0%

`echo "Thank you" + "for submitting" + "the form";`

0%

`echo "Thank you" . "for submitting" . "the form";`

0%

`echo "Thank you" & "for submitting" & "the form"`

0%



What will the value of the variable 'x' be after the following PHP statement: `$x = 9 >= 9 ? 3 : 5;`

0

0%

3

0%

5

0%

9

0%



How many iterations will the following 'for' loop run given the PHP code: `for ($counter = 1; $counter < 5; $counter++)`?

0

0%

1

0%

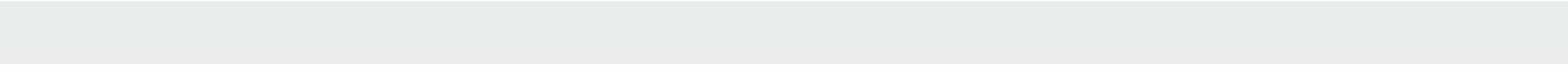
4

0%

5

0%





Summary

- ❑ We have learned how to setup and configure a web server accessible over the internet. The server can host a website, accessible using the world wide web.
- ❑ We have also learned to customise the behaviour of the server, using some of the configuration options, and with our own PHP scripts.
- ❑ So we're done right? Haven't we now achieved our goal?
- ❑ Yes! We have achieved our initial goal. If all we want is a mostly static, small, slow, unsecure website that only our friends will ever find or visit, we could stop here.
- ❑ We can do better though, so let's revise our goal: to create a dynamic, fast, responsive, scalable, secure website on the internet, accessible using the world wide web. In short, an optimised, efficient website.
- ❑ The first improvement is to organise our website to allow it to scale up, to manage a large number of webpages, and potentially multiple users. To do this, we should implement a content management system. We also need to get our data out of the boring XML files, and inject it into our HTML documents – next week.

References

❏ **Web Development and Design Foundations with HTML5 (9e)**

By Terry Felke-Morris

❏ *Chapter 1: Section 1.5*

❏ *Chapter 10: Section 10.3, 10.4, 10.5*

❏ **Programming the World Wide Web (8e)**

By Robert Sebesta

❏ *Chapter 9*